Task: "Event Manager Dashboard"

Build a simple Event Manager Dashboard where users can create, view, and delete events.

Tech Stack Requirements

- **Frontend:** TypeScript + Next.js (App Router, latest version)
- **Backend:** Express.js (Node.js) with **PostgreSQL**
- **Database Access:** Use **raw SQL queries only** (No ORMs like Prisma or Sequelize)
- **Architecture:** Keep **frontend** and **backend** in **separate folders**
- **Communication:** REST API
- **Authentication:** Not required (optional bonus)
- **Design:** Minimal styling, use of Tailwind CSS or any component library like ShadCN

Core Features to Implement

📌 Frontend (Next.js)

1. **Create Event Form**
    - Fields: Event Name, Description, Date, Location
    - Submit to backend via `POST /api/events`
2. **List Events**
    - Fetch and display all events from the backend
    - Show details: ID, Name, Description, Date, Location
3. **Delete Event**
    - Each event should have a delete button
    - Calls `DELETE /api/events/:id` and updates the UI
4. **Event By ID**
    - Fetch Event Details from by ID
    - Provision to Apply to event
5. **Dashboard**
    - The event Owner can See The participants of the event
    - He can cancel the registration of the participant with reason

---

Backend (Express + PostgreSQL)

1. **Endpoints**
    - `POST /api/events` – Create a new event
    - `GET /api/events` – Fetch all events
    - `DELETE /api/events/:id` – Delete event by ID

PostgreSQL Table
sql
CopyEdit

```
CREATE TABLE events (
  id SERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  description TEXT,
  date DATE NOT NULL,
  location TEXT
);
```

2. **Database Access**
    - Use raw SQL queries only

Backend Project Structure

- Model- Queries will be written here
- Controller - Controller code
- Route- Router code

Bonus Points (Optional Features)

You don't have to do these, but they'll earn you extra credit:

1. ✅ **Form Validation**
    - Use `zod` or any schema validator
    - Prevent invalid event submissions
2. 🔍 **Filtering Events**
    - Add filters on the frontend to:
        - Search by name
        - Filter by date or location
3. 🗓️ **Sort Events**
    - Sort events by date ascending or descending
4. ♻️ **Edit Event**
    - Implement `PUT /api/events/:id`

    - Add edit functionality to the frontend form
5. 🧪 **API Error Handling**
    - Handle DB errors gracefully on the backend
    - Show meaningful error messages on the frontend
6. 🔐 **Authentication (Bonus of Bonus)**
    - Add basic email/password login and session-based or JWT-based authentication

---

Submission Guidelines

- Submit A single GitHub repo with a frontend and a backend folder
- Deploy the Project on AWS and configure Nginx with a domain (if Possible) or just share the Normal default link
- Detailed Readme File in the GitHub Repo
- Also share a System Design Document with it, as an attachment in email and GitHub repo

Note

- We are aware of Lovable and Bolt and how they develop the UI, so your submission will be disqualified if you are found developing the complete UI from them
- You are allowed to use those tools to develop 20-30% of the UI or to take inspiration from them

Submission Points

- Code Quality and Readability
- System Design and how scalable it is
- Folder Structure
- Is the UI Clean and sleek

Timeline

- 50 hrs After receiving this assignment